OFFICIAL INTERVIEW MASTER GUIDE

Leave Management & Approval System

A comprehensive technical learning guide, architecture manual, workflow analysis, and 20+ interview questions with model answers for Full-Stack Developers.

Stack: Java 17 | Spring Boot 3 | Node.js Express | MySQL 8.0 | Bootstrap 5.3

Architecture: Layered MVC REST API + Event Audit Microservice **Role:** Full-Stack Java Developer

1 The 60-Second Interview Elevator Pitch

How to introduce this project with maximum confidence:

"I designed and engineered an enterprise-grade **Leave Management & Approval System** designed to streamline corporate leave lifecycles with strict Role-Based Access Control (RBAC) between **Employees** and **Administrators**.

The core backend is built on **Java 17 and Spring Boot 3**, utilizing **Spring Data JPA** and **MySQL 8.0** with optimized JPQL queries to eliminate the N+1 select problem. For compliance and historical tracking, I integrated a secondary **Node.js and Express audit microservice** with dual persistence (MongoDB and a resilient local JSON fallback) triggered via non-blocking background threads.

On the frontend, I developed a modern, responsive interface using **HTML5, Bootstrap 5.3, and ES6 JavaScript** with client-side route guards, custom design tokens, and Lottie animations. Key features include date conflict prevention, immutable state transitions, and a personalized rejection feedback loop where employees receive specific admin review replies."

2 System Architecture & Tech Stack Justification

COMPONENT	TECHNOLOGY	ROLE IN SYSTEM	WHY THIS WAS CHOSEN (INTERVIEW RATIONALE)
Primary Backend	Java 17, Spring Boot 3.x, Spring MVC	Port 8080. Core business logic, validation, authentication, and REST APIs.	Strong typing, enterprise scalability, declarative transaction handling (`@Transactional`), and built-in security tooling.
Relational Database	MySQL 8.0 + Spring Data JPA / Hibernate	Port 3306. Stores `users` and `leave_requests` tables with foreign keys.	ACID guarantees, strict relational foreign key constraints, and relational queries for employee and admin relationships.
Audit Microservice	Node.js, Express, MongoDB + JSON fallback	Port 5000. Append-only immutable event audit log (`/log`, `/logs`).	Decouples heavy logging from the transactional database. Document storage suits variable event schemas; non-blocking Node event loop handles fast logging.
Frontend Client	HTML5, Vanilla ES6 JS, Bootstrap 5.3	Port 3000. Interactive responsive portal for Employees and Admins.	Fast page rendering with zero complex build overhead; uses modern ES6 `async/await` and centralized fetch client with session credentials.

Primary Backend Architecture

Follows **Strict Layered Architecture**:
• **Controller:** Handles HTTP mappings, JSON marshaling, role checks.
• **Service:** Business rules, date validation, state checks, `@Transactional`.
• **Repository:** Spring Data JPA with custom JPQL `JOIN FETCH` queries.
• **DTOs:** Protect internal entities and avoid exposing password hashes.

Resilient Inter-Service Communication

Spring Boot calls Node.js via **asynchronous daemon threads**:
• Fires non-blocking HTTP POST requests to `http://localhost:5000/log`.
• Prevents audit service slowdowns from blocking employee responses.
• Node.js writes to MongoDB; if Mongo is offline, it safely saves to `audit_logs.json`.

3 Complete End-to-End Workflow

Phase 1: Authentication & Role-Based Session Initialization

1. User provides email & password at `/employee-login.html` or `/admin-login.html`.
2. Frontend sends `POST /api/auth/login`.
3. Backend hashes input with **BCrypt** and matches with database password hash.
4. On success, user ID and Role are saved in `HttpSession`; a secure cookie `JSESSIONID` is sent to the browser.
5. Frontend route guard (`auth.js`) verifies session with `GET /api/auth/me` and guards protected routes.

Phase 2: Leave Application (Employee Flow)

1. Employee navigates to **Apply Leave** form.
2. Client enforces validation: dates cannot be in the past; `toDate` must be equal to or after `fromDate`.
3. Form dispatches `POST /api/leaves` with `credentials: 'include'`.
4. Spring Boot verifies role is `EMPLOYEE`, validates payload, and saves record with initial status `PENDING`.
5. Asynchronous background thread dispatches event `LEAVE_APPLIED` to the Node.js audit microservice.
6. Employee dashboard updates live leave summary cards.

Phase 3: Decision & Rejection Feedback Loop (Admin Flow)

1. Administrator logs into **Admin Dashboard**.
2. Backend loads all records using `JOIN FETCH` to pull applicant data and decision metadata in a single query.
3. **Approval:** Admin clicks 'Approve' → dispatches `PUT /api/leaves/{id}/approve` → updates status to `APPROVED`, stamps `decided_at` and `decided_by`.
4. **Rejection:** Admin clicks 'Reject' → modal opens → admin types custom reply reason → dispatches `PUT /api/leaves/{id}/reject` with reason body.
5. System verifies current status is `PENDING`; once finalized, decisions become immutable (HTTP 409 Conflict check).
6. Employee views **Leave History** and instantly sees the dedicated red banner with the Admin's custom feedback.

4 Key Code Concepts & Methods

1. JPQL JOIN FETCH (Solving Hibernate N+1 Problem)

Located in `com.company.leave.repository.LeaveRepository`. Fetches employee user and deciding admin in one SQL JOIN:

```
@Query("SELECT l FROM LeaveRequest l " +
       "JOIN FETCH l.user " +
       "LEFT JOIN FETCH l.decidedBy " +
       "WHERE l.user.id = :userId ORDER BY l.appliedAt DESC")
List<LeaveRequest> findByUserIdWithDetails(@Param("userId") Long userId);
```

2. Asynchronous Audit Client (Decoupled Background Thread)

Located in `com.company.leave.service.AuditServiceClient`. Dispatches fire-and-forget HTTP call without blocking:

```
public void logAction(Long userId, String action, Long leaveId, String details) {
    Thread thread = new Thread(() -> {
        try {
            AuditLogDto payload = new AuditLogDto(userId, action, leaveId, LocalDateTime.now(), details);
            restClient.post().uri(auditUrl).contentType(MediaType.APPLICATION_JSON)
                      .body(payload).retrieve().toBodilessEntity();
        } catch (Exception ex) {
            log.warn("Audit service offline: {}", ex.getMessage());
        }
    });
    thread.setDaemon(true);
    thread.start();
}
```

3. Dual-Persistence Fallback in Node.js Audit Microservice

Located in `backend/audit-service/server.js`. Protects against database downtime:

```
// Attempts MongoDB insertion; if disconnected, falls back safely to local JSON file
try {
  if (isMongoConnected && mongoCollection) {
    const result = await mongoCollection.insertOne({ ...logEntry });
  }
} catch (err) {
  console.warn('MongoDB insert failed, using JSON file fallback:', err.message);
}
writeLocalLog(logEntry); // Always guarantees an on-disk audit record
```

5 20 Essential Interview Questions & Answers

Q1 Can you explain the overall architecture of this application?

Answer: It is a decoupled full-stack architecture. The primary backend is built on Java 17 and Spring Boot 3 with a layered pattern (Controller, Service, Repository, DTO). It connects to MySQL 8.0 via Spring Data JPA. For non-blocking compliance tracking, we implemented a secondary Node.js/Express audit microservice backed by MongoDB and a JSON file fallback. The frontend is a responsive Single/Multi-page interface using HTML5, Bootstrap 5.3, and ES6 JavaScript that communicates with the backend via RESTful APIs and session cookies.

Q2 Why did you use Spring Boot instead of standard Java Servlets or plain Spring?

Answer: Spring Boot drastically reduces boilerplate configuration through auto-configuration, includes an embedded Tomcat server, and simplifies dependency management with starter POMs (`spring-boot-starter-web`, `spring-boot-starter-data-jpa`). It also provides enterprise features such as declarative transactions (`@Transactional`), Bean Validation, and centralized error handling out of the box.

Q3 What is the Hibernate N+1 problem and how did you resolve it?

Answer: The N+1 problem occurs when fetching an entity with lazy relationships (like `LeaveRequest` referencing `User`). Hibernate fires 1 initial query for the leave requests, plus N individual queries to fetch the associated user for each row. I resolved this by writing explicit JPQL queries with `JOIN FETCH l.user LEFT JOIN FETCH l.decidedBy` in `LeaveRepository`. This instructs Hibernate to fetch all parent and child entities in a single SQL query using SQL joins.

Q4 How is authentication handled securely?

Answer: We use BCrypt password hashing via Spring Security's `PasswordEncoder`. Passwords are never stored in plaintext. On successful login, the server establishes an HTTP session (`HttpSession`), storing user ID and role, and issues a `JSESSIONID` cookie to the browser. Every subsequent request passes this cookie with `credentials: 'include'`, allowing server-side role validation without passing sensitive user credentials in headers.

Q5 How is Role-Based Access Control (RBAC) enforced?

Answer: RBAC is enforced both on the server and on the client:
• **Backend:** `LeaveController` invokes `requireRole(session, Role.ADMIN)` or `requireRole(session, Role.EMPLOYEE)`. If the session is missing or the role does not match, it throws an `UnauthorizedException` (401) or `ForbiddenException` (403).
• **Frontend:** The `requireAuth(expectedRole)` guard in `auth.js` verifies the session against `/api/auth/me`. If unauthorized, it immediately redirects the browser to the appropriate login page.

Q6 Why did you use DTOs (Data Transfer Objects) instead of returning Entities?

Answer: Three main reasons:
1. **Security:** The `User` JPA entity contains the BCrypt password hash. Returning `UserDto` ensures password hashes are never serialized over HTTP.
2. **Decoupling:** Changes to the internal database schema don't break the external REST API contract.
3. **Prevent Circular References:** Bidirectional JPA relationships can cause infinite recursion during Jackson JSON serialization; DTOs eliminate this risk.

Q7 How does the application handle exceptions and error responses?

Answer: We implemented a centralized `GlobalExceptionHandler` annotated with `@RestControllerAdvice`. It captures custom exceptions like `ResourceNotFoundException`, `ConflictException`, and validation errors (`MethodArgumentNotValidException`) and maps them to clean HTTP status codes (404, 409, 400). It returns a consistent JSON payload containing `timestamp`, `status`, `error`, and `message`.

Q8 What is `@Transactional` and what benefits does it provide?

Answer: `@Transactional` defines transaction boundaries around service methods. If an exception occurs during leave submission or status changes, all database modifications within the method are rolled back automatically. For read operations, `@Transactional(readOnly = true)` disables Hibernate dirty-checking, boosting query performance and reducing memory overhead.

Q9 Why build an asynchronous audit service instead of calling it synchronously?

Answer: Synchronous network calls to third-party services create latency bottlenecks and tight coupling. If the audit service experiences latency or downtime, a synchronous call would block the employee's request and potentially trigger a 500 Internal Server Error. By dispatching the audit call in a separate daemon thread, the primary transaction completes in milliseconds and the audit system remains completely decoupled.

Q10 How does the Node.js audit microservice achieve fault tolerance?

Answer: On boot, the service checks MongoDB availability with a 2-second timeout. If MongoDB is unreachable or crashes during operation, it logs a warning and stores entries in a local `audit_logs.json` file. When audit logs are requested via `/logs`, it checks MongoDB first, and if offline, reads from the local JSON file. This ensures 100% data retention even during database outages.

Q11 **What business validations did you implement for leave requests?**

Answer:
1. **Date Logic:** `toDate` must be greater than or equal to `fromDate`. Past dates are blocked on the client date pickers and checked in the service.
2. **Character Limits:** Reason input is capped at 500 characters with live character counter.
3. **State Immutability:** Only requests with status `PENDING` can be approved or rejected. Attempting to modify an already `APPROVED` or `REJECTED` leave throws a `ConflictException` (409 Conflict).

Q12 **How does the Admin Rejection feedback loop work?**

Answer: When an Admin rejects a leave, a Bootstrap modal collects a customized rejection reason. The frontend calls `PUT /api/leaves/{id}/reject` with the reason in the request body. The service persists this reason in the `rejection_reason` column in MySQL and records it in the audit log. In the Employee portal, the Leave History table renders a highlighted feedback badge containing the admin's exact response.

Q13 **How did you configure CORS and handle session cookies between ports?**

Answer: Because the frontend runs on port 3000 and the backend on port 8080, CORS was configured in `WebConfig.java` by implementing `WebMvcConfigurer`. We specified `.allowedOriginPatterns("http://localhost:*")` and crucially enabled `.allowCredentials(true)`. In frontend fetch requests (`api.js`), we included `credentials: 'include'`. This enables the browser to send and receive the `JSESSIONID` cookie across different ports.

Q14 **How would you scale this application to 100,000 active employees?**

Answer:
1. **Distributed Session Store:** Migrate from local in-memory `HttpSession` to **Spring Session with Redis** so backend instances can scale horizontally behind an NGINX load balancer.
2. **Event Message Broker:** Replace the thread-based HTTP audit call with **RabbitMQ or Apache Kafka** for guaranteed delivery, message queuing, and replayability.
3. **Database Optimization:** Create composite indexes on `(user_id, status)` and `(applied_at)` in MySQL, and implement Read Replicas for reporting queries.

Q15 **What was the most challenging technical bug you solved?**

Answer: Session cookie loss across origins during local development. Initially, API calls succeeded, but each subsequent call created a new session because `JSESSIONID` was rejected by the browser. I diagnosed that using wildcard `allowedOrigins("*")` alongside `allowCredentials(true)` is prohibited by modern CORS security standards. Resolving this required configuring explicit origin patterns (`allowedOriginPatterns`) on the backend and setting `credentials: 'include'` on the frontend fetch client.

Q16 **What are the differences between `@RestController` and `@Controller`?**

Answer: `@Controller` is standard Spring MVC designed to return HTML view names resolved by a View Resolver (like Thymeleaf). `@RestController` is a convenience annotation that combines `@Controller` and `@ResponseBody`. It automatically serializes return values directly into JSON or XML in the HTTP response body, which is standard for RESTful APIs.

Q17 What HTTP status codes do your APIs return?

Answer:
- **200 OK:** Successful retrieval or update.
- **201 Created:** Successful leave submission.
- **400 Bad Request:** Validation failure (e.g., end date before start date).
- **401 Unauthorized:** Missing or expired session.
- **403 Forbidden:** Valid session but insufficient role privilege (e.g., employee accessing admin endpoint).
- **404 Not Found:** Leave request or user not found.
- **409 Conflict:** Attempting to approve/reject an already decided leave.

Q18 What is Bean Validation and where did you use it?

Answer: Bean Validation (Jakarta Validation) provides declarative constraints on Java fields. In `LeaveApplyRequest` and `LoginRequest`, I used annotations like `@NotBlank`, `@NotNull`, and `@Email`. The controller method arguments are marked with `@Valid`. If constraints are violated, Spring automatically throws `MethodArgumentNotValidException`, which our `GlobalExceptionHandler` formats into user-friendly field-level error messages.

Q19 Why did you build the UI with Vanilla ES6 instead of React or Angular?

Answer: For an internal enterprise dashboard of this scope, Vanilla ES6 JavaScript provides maximum performance, zero build dependencies, rapid deployment, and minimal client-side memory footprint. By structuring the frontend into modular controllers (`api.js`, `auth.js`, `ui.js`), we achieved clean separation of concerns without the overhead of heavy framework compilation pipelines.

Q20 If an employee bypasses the UI and hits the Approve endpoint directly with Postman, what happens?

Answer: The request fails with HTTP 403 Forbidden. The backend endpoint executes `requireRole(session, Role.ADMIN)` before any business logic runs. Because the user's active session is assigned to `EMPLOYEE`, the system halts execution immediately. All security rules are strictly enforced at the backend controller and service layers, never solely on the frontend.

6 Quick Recall Cheat Sheet & Final Interview Tips

Ports & Services

- **Frontend:** Port 3000 (Node static server)
- **Spring Boot:** Port 8080 (Base: `/api`)
- **Audit Microservice:** Port 5000 (`/log`, `/logs`)
- **MySQL Database:** Port 3306 (`leave_management`)

Default Demo Accounts

- **Admin:** `adminajay@gmail.com` / `ajay 1234`
- **Employee:** `ajaydeveloper@gmail.com` / `ajay1234`
- Both stored as secure BCrypt hashes in MySQL.

Key Words to Drop in Tomorrow's Interview

"Separation of Concerns" • "Hibernate N+1 Query Prevention" • "JPQL JOIN FETCH" • "BCrypt Hashing with Salt" • "Role-Based Access Control (RBAC)" • "DTO Data Masking" • "Asynchronous Non-Blocking Execution" • "Graceful Degradation / Fault Tolerance" • "ACID Compliance & @Transactional"